УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 15

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ ДЛЯ МАНИПУЛИРОВАНИЯ URL-АДРЕСОМ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейших программ для манипулирования URL-адресом

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Взаимодействие с Интернетом

Интернет прочно вошел в нашу жизнь. Очень часто необходимо передать информацию на Web-сервер или, наоборот, получить с него какие-либо данные, например, котировки валют или прогноз погоды, проверить наличие писем в почтовом ящике и т. д. В состав стандартной библиотеки Python входит множество модулей, позволяющих работать практически со всеми протоколами Интернета. В последующих шагах рассмотрим только наиболее часто встречающиеся задачи:

- разбор URL-адреса и строки запроса на составляющие;
- преобразование гиперссылок;
- разбор HTML-эквивалентов;
- определение кодировки документа;
- обмен данными по протоколу HTTP с помощью модулей http.client и urllib.request.

4.2 Разбор URL-адреса

С помощью модуля **urllib.parse** можно манипулировать **URL**-адресом – например, разобрать его на составляющие или получить абсолютный **URL**-адрес, указав базовый и относительный адреса. **URL**-адрес (его также называют интернет-адресом) состоит из следующих элементов:

<Протокол>://<Домен>:<Порт>/<Путь>;<Параметры>?<Запрос>#<Якорь>

СхемаURL-адреса для протокола FTP выглядит по-другому:

<Протокол>://<Пользователь>:<Пароль>@<Домен>

Разобрать URL-адрес на составляющие позволяет функция urlparse():

urlparse (<URL-адрес>[<Схема>[, <Разбор якоря>]])

Функция возвращает объект **ParseResult** с результатами разбора **URL**-адреса. Получить значения можно с помощью атрибутов или индексов. Объект можно преобразовать в кортеж из следующих элементов: **(scheme, netloc, path, params, query, fragment)**. Элементы соответствуют схеме **URL**-адреса:

<scheme>://<netloc>/<path>;<params>?<query>#<fragment>.

Обратите внимание на то, что название домена, хранящееся в атрибуте **netloc**, будет содержать номер порта. Кроме того, не ко всем атрибутам объекта можно получить доступ с помощью индексов. Пример кода, разбирающего **URL**-адрес, приведен ниже.

```
from urllib.parse  import urlparse
url = urlparse("http://www.admin.ru:80/test.php;st?var=5#metka")
url
ParseResult(scheme='http', netloc='www.admin.ru:80', path='/test.php',
params='st', query='var=5', fragment='metka')
```

Во втором параметре функции urlparse() можно указать название протокола, которое будет использоваться, если таковой отсутствует в составе URL-адреса. По умолчанию это пустая строка.

Примеры:

```
urlparse ("//www.admin.ru/test.php")
ParseResult(scheme='', netloc='www.admin.ru', path='/test.php',
params='', query='', fragment='')
urlparse ("//www.admin.ru/test.php", "http")
ParseResult(scheme='http', netloc='www.admin.ru', path='/test.php',
params='', query='', fragment='')
```

Объект ParseResult, возвращаемый функцией urlparse(), содержит следующие атрибуты:

– **scheme** - название протокола. Значение доступно также по индексу 0. По умолчанию - пустая строка.

Пример:

```
url.scheme, url[0]
('http', 'http')
```

– **netloc** - название домена вместе с номером порта. Значение доступно также по индексу 1. По умолчанию - пустая строка.

Пример:

```
url.netloc, url[1]
('www.admin.ru:80', 'www.admin.ru:80')
```

– **hostname** - название домена в нижнем регистре. Значение по умолчанию – **None**;

– **port** - номер порта. Значение по умолчанию – **None**.

Пример:

```
url.hostname, url.port
('www.admin.ru', 80)
```

– **path** - путь. Значение доступно также по индексу 2. По умолчанию – пустая строка.

Пример:

```
url.path, url[2]
('/test.php', '/test.php')
```

– **params** – параметры. Значение доступно также по индексу 3. По умолчанию - пустая строка.

Пример:

```
url.params, url[3]
('st', 'st')
```

– **query** – строка запроса. Значение доступно также по индексу 4. По умолчанию - пустая строка.

Пример:

```
url.query, url[4]
('var=5', 'var=5')
```

– **fragment** – якорь. Значение доступно также по индексу 5. По умолчанию – пустая строка.

Пример:

```
url.fragment, url[5]
('metka', 'metka')
```

Если третий параметр в функции urlparse() имеет значение False, то якорь будет входить в состав значения других атрибутов (обычно хранящего предыдущую часть адреса), а не fragment. По умолчанию параметр имеет значение True.

Примеры:

```
u = urlparse("http://site.ru/add.php?v=5#metka")
u.query, u.fragment
('v=5', 'metka')
u = urlparse("http://site.ru/add.php?v=5#metka", "", False)
u.query, u.fragment
('v=5#metka', '')
```

– username- имя пользователя. Значение по умолчанию – None;

– password - пароль. Значение по умолчанию – None.

Пример:

```
ftp = urlparse("ftp://user:123456@mysite.ru")
ftp.scheme, ftp.hostname, ftp.username, ftp.password
('ftp', 'mysite.ru', 'user', '123456')
```

Метод geturl() возвращает изначальный URL-адрес.

Пример:

```
url.geturl()
'http://www.admin.ru:80/test.php;st?var=5#metka'
```

Выполнить обратную операцию (собрать URL-адрес из отдельных значений) позволяет функция urlunparse (<Последовательность>**)**.

```
from urllib.parse import urlunparse
t = ('http', 'www.admin.ru:80', '/test.php', '', 'var=5', 'metka')
urlunparse (t)
'http://www.admin.ru:80/test.php?var=5#metka'
l = ['http', 'www.admin.ru:80', '/test.php', '', 'var=5', 'metka']
```

```
urlunparse (l)
'http://www.admin.ru:80/test.php?var=5#metka'
```

Вместо функции urlparse() можно воспользоваться функцией urlsplit (<URL-адрес>[, <Схема> [, <Разбор якоря>]]). Ее отличие от urlparse() проявляется в том, что она не выделяет из интернет-адреса параметры. Функция возвращает объект SplitResuit с результатами разбора URL-адреса. Объект можно преобразовать в кортеж из следующих элементов: (scheme, netloc, path, query, fragment). Обратиться к значениям можно как по индексу, так и по названию атрибутов. Пример использования функции urlsplit() приведен ниже.

```
from urllib.parse import  urlsplit
url = urlsplit("http://www.admin.ru:80/test.php;st?var=5#metka")
url
SplitResult(scheme='http', netloc='www.admin.ru:80', path='/test.php;st',
query='var=5', fragment='metka')
url[0], url[1], url[2], url[3], url[4]
('http', 'www.admin.ru:80', '/test.php;st', 'var=5', 'metka')
url.scheme, url.netloc, url.hostname, url.port
('http', 'www.admin.ru:80', 'www.admin.ru', 80)
url.path, url.query, url.fragment
('/test.php;st', 'var=5', 'metka')
ftp = urlsplit("ftp://user:123456@mysite.ru")
ftp.scheme, ftp.hostname, ftp.username, ftp.password
('ftp', 'mysite.ru', 'user', '123456')
```

Выполнить обратную операцию (собрать **URL**-адрес из отдельных значений) позволяет функция **urlunsplit (<Последовательность>)**.

```
from urllib.parse import urlunsplit
t = ('http', 'www.admin.ru:80', '/test.php;st', 'var=5', 'metka')
urlunsplit(t)
'http://www.admin.ru:80/test.php;st?var=5#metka'
```

4.3 Кодирование и декодирование строки запроса

На предыдущем шаге научились разбирать URL-адрес на составляющие. Обратите внимание на то, что значение параметра <Запрос> возвращается в виде строки. Строка запроса является составной конструкцией, содержащей пары параметр=значение. Все специальные символы внутри названия параметра и значения кодируются последовательностями %nn. Например, для параметра str, имеющего значение "Строка" в кодировке Windows-1251, строка запроса будет выглядеть так:

```
str=%Dl%F2%F0%EE%EA%E0
```

Если строка запроса содержит несколько пар **параметр=значение**, то они разделяются символом **&**. Добавим параметр **v** со значением 10:

```
str=%Dl%F2%F0%EE%EA%E0&v=10
```

В строке запроса может быть несколько параметров с одним названием, но разными значениями, - например, если передаются значения нескольких выбранных пунктов в списке с множественным выбором:

str=%Dl%F2%F0%EE%EA%EG&v=10&v=20

Разобрать строку запроса на составляющие и декодировать данные позволяют следующие функции из модуля **urllib.parse**:

- **parse_qs ()** – разбирает строку запроса и возвращает словарь с ключами, представляющими собой названия параметров, и значениями, которыми станут значения этих параметров. Формат функции:

parse_qs (<Строка запроса> [, keep_blank_values=False] [, strict_parsing=False] [, encoding='utf-8'] [, errors='replace'])

Если в параметре **keep_blank_values** указано значение **True**, то параметры, не имеющие значений внутри строки запроса, также будут добавлены в результат. По умолчанию пустые параметры игнорируются. Если в параметре **strict_parsing** указано значение **True**, то при наличии ошибки возбуждается исключение **ValueError**. По умолчанию ошибки игнорируются. Параметр **encoding** позволяет указать кодировку данных, а параметр **errors** - уровень обработки ошибок. Пример разбора строки запроса:

```
from urllib.parse  import parse_qs
s = "str=%D1%F2%F0%EE%EA%E0&v=10&v=20&t="
parse_qs(s, encoding="cp1251")
{'v': ['10', '20'], 'str': ['Строка']}
parse_qs(s, keep_blank_values=True, encoding="cp1251")
{'v': ['10', '20'], 'str': ['Строка'], 't': ['']}
```

- **parse_qsl ()** – функция аналогична **parse_qs()**, только возвращает не словарь, а список кортежей из двух элементов: первый элемент "внутреннего" кортежа содержит название параметра, а второй элемент - его значение. Если строка запроса содержит несколько параметров с одинаковыми значениями, то они будут расположены в разных кортежах. Формат функции:

parse_qsl (<Строка запроса> [, keep_blank_values=False] [, strict_parsing=False] [, encoding='utf-8'] [, errors='replace'])

Пример разбора строки запроса:

```
from urllib.parse  import parse_qsl
s = "str=%D1%F2%F0%EE%EA%E0&v=10&v=20&t="
parse_qsl(s, encoding="cp1251")
[('str', 'Строка'), ('v', '10'), ('v', '20')]
parse_qsl(s, keep_blank_values=True, encoding="cp1251")
[('str', 'Строка'), ('v', '10'), ('v', '20'), ('t', '')]
```

Выполнить обратную операцию - преобразовать отдельные составляющие в строку запроса - позволяет функция **urlencode()**. Формат функции:

urlencode (<Объект> [, doseq=False] [, safe=''] [, encoding=None] [, errors=None])

В качестве первого параметра можно указать словарь с данными или последовательность, каждый элемент которой содержит кортеж из двух элементов: первый элемент такого кортежа станет параметром, а второй элемент - его

значением. Параметры и значения автоматически обрабатываются с помощью функции **quote_plus()** из модуля **urllib.parse**. В случае указания последовательности параметры внутри строки будут идти в том же порядке, что и внутри последовательности. Пример указания словаря и последовательности приведен ниже.

```
from urllib.parse import urlencode
params = {"str": "Строка 2", "var": 20} # Словарь
urlencode (params, encoding="cp1251")
'var=20&str=%D1%F2%F0%EE%EA%E0+2'
params = [ ("str", "Строка 2"), ("var", 20) ] # Список
urlencode (params, encoding="cp1251")
'str=%D1%F2%F0%EE%EA%E0+2&var=20'
```

Если необязательный параметр **doseq** в функции **urlencode()** имеет значение **True**, то во втором параметре кортежа можно указать последовательность из нескольких значений. В этом случае в строку запроса добавляются несколько параметров со значениями из этой последовательности. Значение параметра **doseq** по умолчанию - **False**. В качестве примера укажем список из двух элементов.

```
params = [ ("var", [10, 20]) ]
urlencode(params, doseq=False, encoding="cp1251")
'var=%5B10%2C+20%5D'
urlencode(params, doseq=True, encoding="cp1251")
'var=10&var=20'
```

Последовательность также можно указать в качестве значения в словаре:

```
params = { "var": [10, 20] }
urlencode(params, doseq=True, encoding="cp1251")
'var=10&var=20'
```

Выполнить кодирование и декодирование отдельных элементов строки запроса позволяют следующие функции из модуля **urllib.parse**:

– **quote ()** – заменяет все специальные символы последовательностями **%nn**. Цифры, английские буквы и символы подчеркивания (_), точки (.) и дефиса (-) не кодируются. Пробелы преобразуются в последовательность **%20**. Формат функции:

```
quote (<Строка>[, safe='/'][, encoding=None][, errors=None])
```

В параметре **safe** можно указать символы, которые преобразовывать нельзя, – по умолчанию параметр имеет значение /. Параметр **encoding** позволяет указать кодировку данных, а параметр **errors** - уровень обработки ошибок.

Пример:

```
from urllib.parse import quote
quote("Строка", encoding="cp1251") # Кодировка Windows-1251
'%D1%F2%F0%EE%EA%E0'
quote("Строка", encoding="utf-8")  # Кодировка UTF-8
'%D0%A1%D1%82%D1%80%D0%BE%D0%BA%D0%B0'
quote("/~nik/"), quote("/~nik/", safe="")
('/%7Enik/', '%2F%7Enik%2F')
```

```
quote("/~nik/", safe="/~")
'/~nik/'
```

– **quote_plus ()** – функция аналогична **quote()**, но пробелы заменяются символом +, а не преобразуются в последовательность **%20**. Кроме того, по умолчанию символ / преобразуется в последовательность **%2F**. Формат функции:

```
quote_plus (<Строка>[, safe=''][, encoding=None][, errors=None])
```

Примеры:

```
from urllib.parse import quote, quote_plus
quote("Строка 2", encoding="cp1251")
'%D1%F2%F0%EE%EA%E0%202'
quote_plus("Строка 2", encoding="cp1251")
'%D1%F2%F0%EE%EA%E0+2'
quote_plus("/~nik/")
'%2F%7Enik%2F'
quote_plus("/~nik/", safe="/~")
'/~nik/'
```

– **quote_from_bytes ()** – функция аналогична **quote()**, но в качестве первого параметра принимает последовательность байтов, а не строку. Формат функции:

```
quote_from_bytes (<Последовательность байтов>[, safe='/'])
```

Пример:

```
from urllib.parse import quote_from_bytes
quote_from_bytes(bytes("Строка 2", encoding="cp1251"))
'%D1%F2%F0%EE%EA%E0%202'
```

– **unquote ()** – заменяет последовательности **%nn** соответствующими символами. Символ + пробелом не заменяется. Формат функции:

```
unquote (<Строка> [, encoding = 'utf-8'][, errors = 'replace'])
```

Примеры:

```
from urllib.parse import unquote
unquote ("%D1%F2%F0%EE%EA%E0", encoding="cp1251")
'Строка'
unquote ('%D1%F2%F0%EE%EA%E0+2', encoding="cp1251")
'Строка+2'
```

– **unquote_plus ()** – функция аналогична **unquote()**, но дополнительно заменяет символ + пробелом. Формат функции:

```
unquote_plus (<Строка> [, encoding = 'utf-8'][, errors = 'replace'])
```

Примеры:

```
from urllib.parse import unquote_plus
unquote_plus("%D1%F2%F0%EE%EA%E0+2", encoding="cp1251")
'Строка 2'
unquote_plus("%D1%F2%F0%EE%EA%E0%202", encoding="cp1251")
'Строка 2'
```

– **unquote_to_bytes ()** – функция аналогична **unquote()**, но в качестве первого параметра принимает строку или последовательность байтов и возвращает последовательность байтов. Формат функции:

```
unquote_to_bytes (<Строка или последовательность байтов>)
```

Примеры:

```
from urllib.parse import unquote_to_bytes
unquote_to_bytes("%D1%F2%F0%EE%EA%E0%202")
b'\xd1\xf2\xf0\xee\xea\xe0 2'
unquote_to_bytes(b"%D1%F2%F0%EE%EA%E0%202")
b'\xd1\xf2\xf0\xee\xea\xe0 2'
unquote_to_bytes("%D0%A1%D1%82%D1%80%D0%BE%D0%BA%D0%B0")
b'\xd0\xa1\xd1\x82\xd1\x80\xd0\xbe\xd0\xba\xd0\xb0'
str(_,"utf-8")
'Строка'
```

4.4 Преобразование относительного URL-адреса в абсолютный

Очень часто в коде **Web**-страниц указываются не абсолютные **URL**-адреса, а относительные. При относительном **URL**-адресе путь определяется с учетом местоположения страницы, на которой находится ссылка, или значения параметра **href** тега **<base>**. Преобразовать относительную ссылку в абсолютный **URL**-адрес позволяет функция **urljoin()** из модуля **urllib.parse**. Формат функции:

urljoin(<Базовый URL-адрес>, <Относительный или абсолютный URL-адрес> [, <Разбор якоря>])

Для примера рассмотрим преобразование различных относительных интернет-адресов.

```
from urllib.parse import urljoin
urljoin('http://www.admin.ru/fl/f2/test.html', 'file.html')
'http://www.admin.ru/fl/f2/file.html'
urljoin('http://www.admin.ru/f1/f2/test.html', 'f3/file.html')
'http://www.admin.ru/f1/f2/f3/file.html'
urljoin('http://www.admin.ru/f1/f2/test.html', '/file.html')
'http://www.admin.ru/file.html'
urljoin('http://www.admin.ru/f1/f2/test.html', './file.html')
'http://www.admin.ru/f1/f2/file.html'
urljoin('http://www.admin.ru/f1/f2/test.html', '../file.html')
'http://www.admin.ru/f1/file.html'
urljoin('http://www.admin.ru/f1/f2/test.html', '../../file.html')
'http://www.admin.ru/file.html'
urljoin('http://www.admin.ru/f1/f2/test.html', '../../../file.html')
'http://www.admin.ru/../file.html'
```

В последнем случае специально указан уровень относительности больше, чем нужно. Как видно из результата, в таком случае формируется некорректный интернет-адрес.

4.5 Разбор HTML-эквивалентов

В языке **HTML** некоторые символы являются специальными - например, знаки «меньше» (<) и «больше» (>), кавычки и др. Для отображения специальных символов служат так называемые **HTML-эквиваленты**. При этом знак «меньше» заменяется последовательностью **<**, а знак «больше» – **>**. Манипулировать **HTML**-эквивалентами позволяют следующие функции из модуля **xml.sax.saxutils**:

- **escape (<Строка> [, <Словарь>])** - заменяет символы <, > и & соответствующими HTML-эквивалентами. Необязательный параметр **<Словарь>** позволяет указать словарь с дополнительными символами в качестве ключей и их **HTML**-эквивалентами в качестве значений.

Примеры:

```
from xml.sax.saxutils import escape
s = """&<>" """
escape (s)
'&amp;&lt;&gt;" '
escape(s, { " ' " : "&quot;", " ": " " } )
'&amp;&lt;&gt;" '
```

- **quoteattr (<Строка> [, <Словарь>])** – функция аналогична **escape()**, но дополнительно заключает строку в кавычки или апострофы. Если внутри строки встречаются только двойные кавычки, то строка заключается в апострофы. Если внутри строки встречаются и кавычки, и апострофы, то двойные кавычки заменяются **HTML**-эквивалентом, а строка заключается в двойные кавычки. Если кавычки и апострофы не входят в строку, то строка заключается в двойные кавычки.

Примеры:

```
from xml.sax.saxutils import quoteattr
print(quoteattr("""&<>" """))
'&amp;&lt;&gt;" '
print(quoteattr("""&<>"""""))
"&amp;&lt;&gt;&quot;"
print(quoteattr("""&<>" """, {"" : "&quot;" } ))
"&amp;&lt;&gt;&quot; "
```

- **unescape (<Строка> [, <Словарь>])** – заменяет **HTML**-эквиваленты **&**, **<** и **>** обычными символами. Необязательный параметр **<Словарь>** позволяет указать словарь с дополнительными **HTML**-эквивалентам и в качестве ключей и обычными символами в качестве значений.

Примеры:

```
from xml.sax.saxutils import unescape
s = '&amp;&lt;&gt;&quot; '
unescape(s)
'&<>&quot; '
unescape (s, { "&quot;": "", " ": " " } )
'& < > "  '
```

Для замены символов <, > и & **HTML**-эквивалентами также можно воспользоваться функцией **escape (<Строка>[, <Флаг>])** из модуля **html**. Если во втором параметре указано значение **False**, двойные кавычки и апострофы не будут заменяться **HTML**-эквивалентами. А функция **unescape (<Строка>)**, объявленная в том же модуле и поддерживаемая, начиная с **Python 3.4**, выполняет обратную операцию - замену **HTML**-эквивалентов соответствующими им символами.

```
import html
html.escape ("""&<>"' """)
'&amp;&lt;&gt;&quot;&#x27;'
html.escape ("""&<>"' """, False)
'&amp;&lt;&gt;"\' '
html.unescape ('&amp;&lt;&gt;&quot;&#x27; ')
'&<>"\' '
html.unescape ('&amp;&lt;&gt;"\' ')
'&<>"\' '
```

4.6 Пример решения задачи на языке программирования Python

Задача. Дан URL-адрес. Необходимо написать программу, которая разбивает его на составные части и выводит протокол, доменное имя и путь к ресурсу. Для выполнения задачи использовать стандартные средства Python для работы с URL.

Решение задачи:

```
from urllib.parse import urlparse
url = input("Введите URL: ")
parsed_url = urlparse(url)
protocol = parsed_url.scheme
domain = parsed_url.netloc
path = parsed_url.path
print("Протокол:", protocol)
print("Домен:", domain)
print("Путь:", path)
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Напишите программу, которая принимает URL-адрес и выводит протокол, доменное имя и путь к ресурсу.

2 Создайте программу, которая определяет, используется ли в URL защищённый протокол HTTPS, и выводит соответствующее сообщение.

3 Напишите программу, которая выводит все параметры запроса и их значения из заданного URL.

4 Создайте программу, которая добавляет новый параметр lang=ru к URL-адресу, сохраняя все существующие параметры.

5 Напишите программу, которая удаляет все параметры запроса из URL и выводит «очищенный» URL.

6 Создайте программу, которая заменяет доменное имя в URL на example.com, оставляя остальные части без изменений.

7 Напишите программу, которая определяет номер порта в URL. Если порт не указан, выведите сообщение «Порт не задан».

8 Создайте программу, которая разбивает путь URL на отдельные элементы и выводит их построчно.

9 Напишите программу, которая проверяет наличие якоря (#fragment) в URL и выводит его значение или сообщение «Якорь отсутствует».

10 Создайте программу, которая меняет протокол URL: http на https и https на http.

11 Напишите программу, которая подсчитывает количество параметров запроса в URL.

12 Создайте программу, которая формирует URL из отдельных компонентов: протокол, домен, путь и параметры запроса.

13 Напишите программу, которая определяет, содержит ли URL путь к файлу (наличие расширения, например .html, .php, .jpg).

14 Напишите программу, которая проверяет корректность URL (наличие протокола и домена) и выводит «URL корректен» или «URL некорректен».

15 Создайте программу, которая кодирует параметры URL (заменяет пробелы и специальные символы на URL-кодировку).

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Охарактеризуйте функцию с помощью которой можно разобрать URL-адрес на составляющие в языке программирования Python?

7.2 Опишите схему URL-адреса для протокола FTP в языке программирования Python.

7.3 Опишите из каких элементов состоит URL-адрес (его также называют интернет-адресом) в языке программирования Python?

7.4 Опишите атрибуты, которые содержит объект ParseResult, возвращаемый функцией urlparse() в языке программирования Python.

7.5 Опишите функции из модуля urllib.parse, позволяющие разобрать строку запроса на составляющие и декодировать данные в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.